SIT NAGPUR • Dept. of Computer Science • JavaScript Lab Tanmay Sankulwar | PRN: 24070521058

EXPERIMENT 12

Student Name: Tanmay Sankulwar
PRN: 24070521058
File Path: `EXP12/index.html`, `EXP12/script.js`, `EXP12/style.css`, `EXP12/Casestudy/`

Experiment Title

Live Task Manager with Asynchronous AJAX Integration, Local Storage Persistence & Dynamic DOM Operations

Objective

1. To construct a dynamic Single-Page Task Management application using jQuery 3.7.1 and modern JavaScript DOM manipulation techniques.
2. To synchronize client application state with the browser's `localStorage` via JSON serialization (`JSON.stringify()`, `JSON.parse()`) ensuring data survival across sessions.
3. To calculate and display real-time reactive counter metrics reflecting total and completed task counts.
4. To implement asynchronous data ingestion by requesting remote REST API endpoints using jQuery `$.ajax()`.
5. To employ higher-order array methods (`filter()`, `some()`, `forEach()`, `reduce()`) for duplicate prevention and dynamic filtering.
6. To study interactive web portal design and full-fledged e-commerce cart state management through practical case studies.

Software / Tools Required

1. **Operating System:** Windows 10 / 11
2. **Development Environment:** Visual Studio Code / Antigravity IDE
3. **Web Browser:** Google Chrome / Modern Web Browser
4. **Languages & Technologies:** HTML5, Modern CSS3 (Flexbox, Card Shadowing, Responsive UI), Modern JavaScript (ES6+), jQuery 3.7.1, RESTful APIs

Program / Task

Task 12 — Live Task Manager with AJAX Integration

Files: `EXP12/index.html`, `EXP12/script.js`, `EXP12/style.css`

The primary application provides a responsive **Live Task Manager** featuring: - **Interactive Task Addition:** An input form with focus management capturing custom user tasks. - **Dynamic Task Card Rendering:** Uses jQuery to construct task containers containing custom checkboxes, styled labels, and individual red delete buttons. - **Completion Toggling:** Clicking the checkbox toggles the task's boolean `completed` state, applies strike-through styling (`.completed span { text-decoration: line-through; }`), and synchronizes state to `localStorage` . - **Task Deletion:** Clicking **Delete** removes the item from the tasks array using `.filter()` , updates storage, and re-renders the list. - **Real-Time Counter Badge:** Dynamically computes total and completed tasks (`X tasks | Y completed`). - **AJAX Remote Data Fetching:** Clicking **Load Sample Tasks** triggers an asynchronous `$.ajax()` GET request to `https://jsonplaceholder.typicode.com/todos?_limit=5` . Before adding items, the system evaluates `.some()` to filter out duplicates, updates storage, and displays a user feedback message.

Source Code

Main Application Markup (`EXP12/index.html`)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Live Task Manager</title>
    <link rel="stylesheet" href="style.css">
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>

    <div class="container">
        <header>
            <h1>Live Task Manager</h1>
            <p>Organize your tasks and track your progress.</p>
        </header>

        <section class="task-form">
            <h2>Add a New Task</h2>

            <form id="taskForm">
                <input type="text" id="taskInput"
                       placeholder="Enter your task..." required>

                <button type="submit">Add Task</button>
            </form>
        </section>

        <section class="task-section">
            <div class="section-heading">
                <h2>My Tasks</h2>
                <span id="taskCount">0 tasks</span>
            </div>

            <div id="taskList">
                <p class="empty-message">No tasks added yet.</p>
            </div>
        </section>

        <section class="data-section">
            <h2>AJAX Data Integration</h2>
            <p>Load sample tasks from an external data source.</p>
            <button type="button" id="loadTasks">Load Sample Tasks</button>
            <p id="ajaxStatus" aria-live="polite"></p>
        </section>

        <footer>
            <p>JavaScript Practical 12 | Live Task Manager</p>
        </footer>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

Main Application JavaScript (`EXP12/script.js`)

```
$(document).ready(function () {

    let tasks = JSON.parse(localStorage.getItem("tasks")) || [];

    function saveTasks() {
        localStorage.setItem("tasks", JSON.stringify(tasks));
    }

    function updateTaskCount() {
        const total = tasks.length;
        const completed = tasks.filter(task => task.completed).length;

        $("#taskCount").text(
            total + " tasks | " + completed + " completed"
        );
    }

    function displayTasks() {
        $("#taskList").empty();

        if (tasks.length === 0) {
            $("#taskList").html(
                '<p class="empty-message">No tasks added yet.</p>'
            );
            updateTaskCount();
            return;
        }

        tasks.forEach(function (task) {
            const taskItem = $("<div>").addClass("task-item");

            if (task.completed) {
                taskItem.addClass("completed");
            }

            const label = $("<label>");
            const checkbox = $("<input>")
                .attr("type", "checkbox")
                .prop("checked", task.completed)
                .on("change", function () {
                    task.completed = $(this).prop("checked");
                    saveTasks();
                    displayTasks();
                });

            const taskText = $("<span>").text(task.text);

            label.append(checkbox, taskText);

            const deleteButton = $("<button>")
                .addClass("delete-btn")
                .text("Delete")
                .on("click", function () {
                    tasks = tasks.filter(item => item.id !== task.id);
                    saveTasks();
                    displayTasks();
                });

            taskItem.append(label, deleteButton);
            $("#taskList").append(taskItem);
        });
```

```
    updateTaskCount();
}

$("#taskForm").on("submit", function (event) {
    event.preventDefault();

    const taskText = $("#taskInput").val().trim();

    if (taskText === "") {
        return;
    }

    tasks.push({
        id: Date.now() + Math.random(),
        text: taskText,
        completed: false
    });

    saveTasks();
    displayTasks();

    $("#taskInput").val("").focus();
});

$("#loadTasks").on("click", function () {
    $("#ajaxStatus").text("Loading sample tasks...");

    $.ajax({
        url: "https://jsonplaceholder.typicode.com/todos?_limit=5",
        method: "GET",
        dataType: "json",

        success: function (data) {
            let added = 0;

            data.forEach(function (item) {
                const exists = tasks.some(
                    task => task.text === item.title
                );

                if (!exists) {
                    tasks.push({
                        id: Date.now() + Math.random(),
                        text: item.title,
                        completed: item.completed
                    });

                    added++;
                }
            });

            saveTasks();
            displayTasks();

            $("#ajaxStatus").text(
                added + " sample tasks loaded successfully."
            );
        },

        error: function () {
            $("#ajaxStatus").text(
                "Unable to load tasks. Check your internet connection."
            );
        }
    });
```

```
    });

    displayTasks();

});
```

Explanation / Concepts

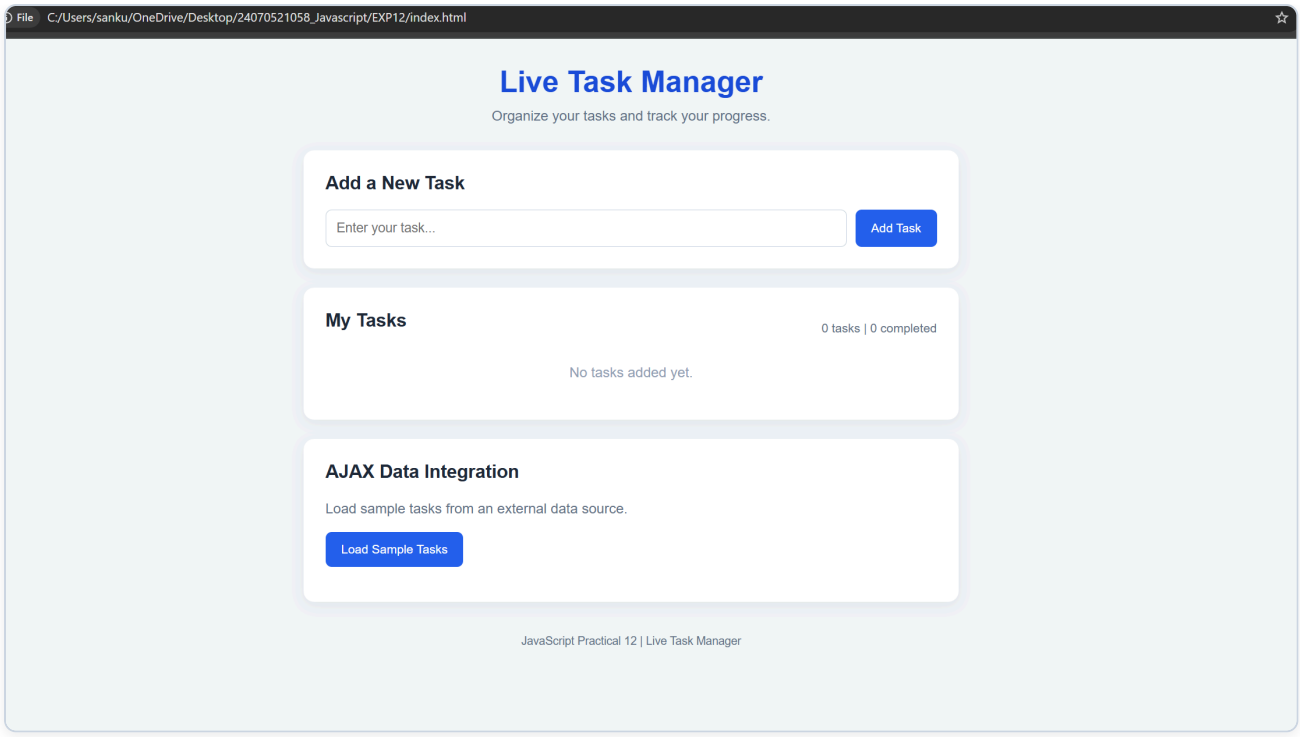
1. **jQuery Event Model & DOM Manipulation:**
2. `$(document).ready()` guarantees execution only after the complete DOM tree is parsed.
3. Elements are dynamically created in memory (`$("<div>")` , `("<label>")` , `("<button>")`), decorated with attributes and classes, and appended using `.append()` .
4. `.empty()` ensures clean list repainting without ghost DOM fragments.
5. **Persistent Web Storage Synchronization:**
6. State initializes with `JSON.parse(localStorage.getItem("tasks")) || []` .
7. Any mutation (addition, checkbox toggle, or deletion) immediately calls `saveTasks()` executing `localStorage.setItem("tasks", JSON.stringify(tasks))` .
8. **Reactive Counter Metrics:**
9. Filter expressions calculate completion status on the fly: `const completed = tasks.filter(task => task.completed).length;`
10. Keeps the header counter in sync with all user interactions.
11. **Asynchronous REST Integration (`$.ajax`):**
12. Configured with `method: "GET"` and `dataType: "json"` .
13. The duplicate prevention check `tasks.some(task => task.text === item.title)` prevents reloading duplicate records if the button is pressed repeatedly.

Output

1. **Task Operations:** Typing a task and pressing Enter or clicking **Add Task** immediately inserts the record, increments the count badge, and resets input focus.
2. **Completion & Deletion:** Marking a checkbox draws a line through the task text and updates the completed count. Clicking **Delete** removes the card immediately.
3. **AJAX Loading:** Clicking **Load Sample Tasks** contacts JSONPlaceholder, ingests up to 5 remote tasks, updates `localStorage` , and provides live textual status confirmation.

Screenshot

Live Task Manager Output



Screenshot 2026-10-26 225929.png

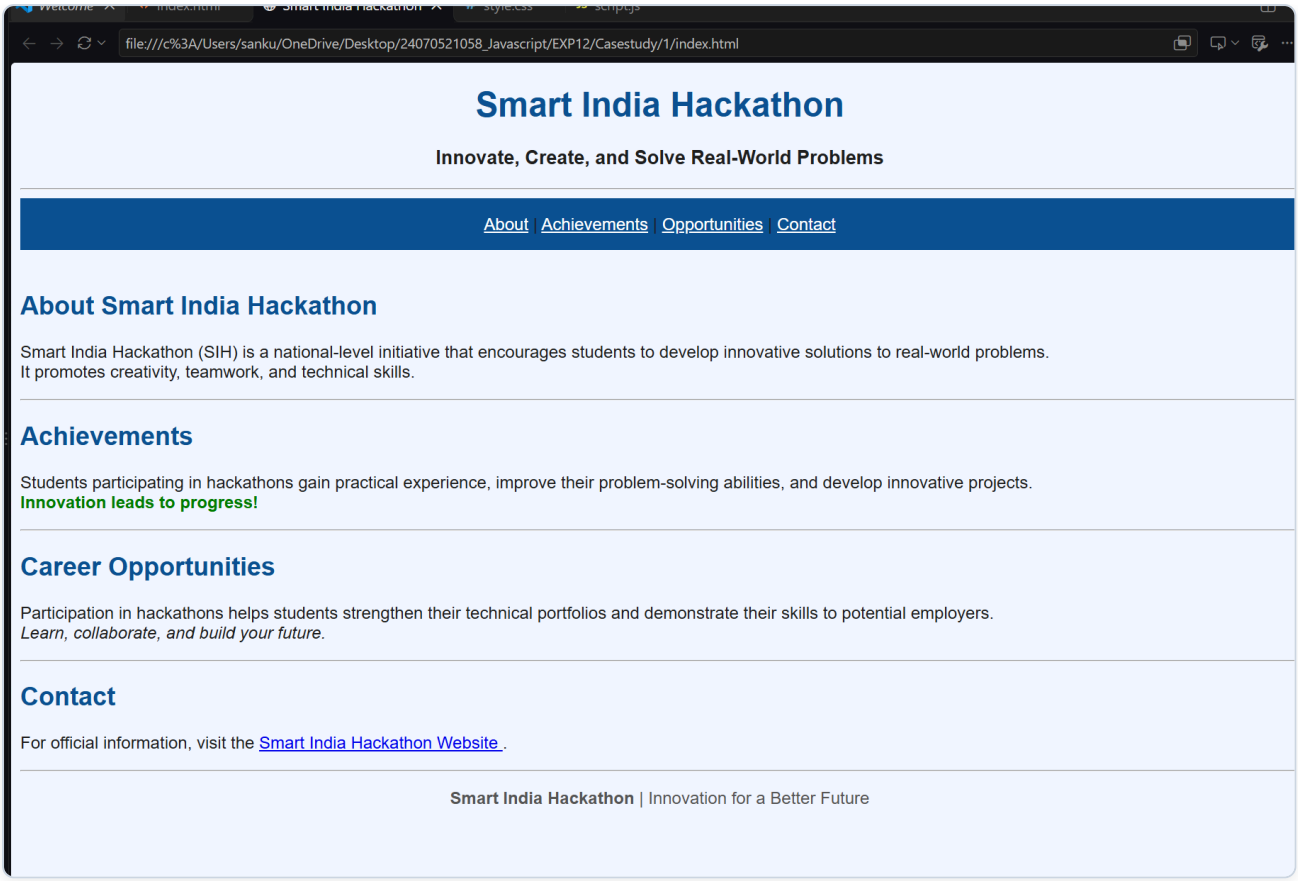
Case Study

Case Study 1 — Smart India Hackathon Informational Portal

Files: `EXP12/Casestudy/1/index.html`, `EXP12/Casestudy/1/script.js`, `EXP12/Casestudy/1/style.css`

The case study provides an academic portal designed for student hackathon awareness: - **Semantic Structure:** Implements internal bookmark navigation (`#about`, `#achievements`, `#placements`, `#contact`). - **Interactive Event Logging:** Intercepts navigation clicks using `DOMContentLoaded` and `addEventListener`, logging section access and triggering welcome alerts.

Case Study 1 Screenshot



Screenshot 2026-10-26 230528.png

Case Study 2 — Bandhan Store: Premium Festive E-Commerce Web Application

Files: `EXP12/Casestudy/2/index.html` , `EXP12/Casestudy/2/script.js` , `EXP12/Casestudy/2/style.css`

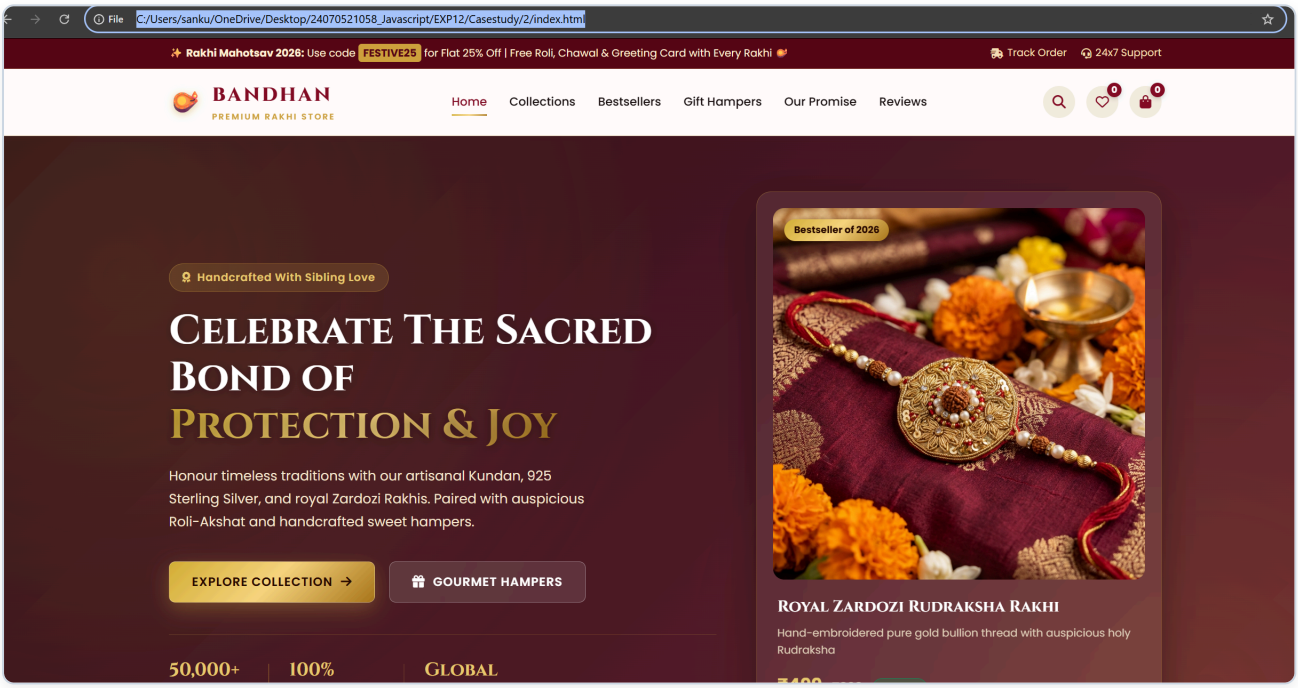
An advanced e-commerce web application featuring comprehensive state management and interactive user experiences: - **Product Catalog & Dynamic Category Filtering:** Array of catalog objects filtered across categories (`Designer & Zardozi` , `Pure Silver` , `Royal Kundan` , `Gift Hampers`). - **Interactive Slide-Over Cart Drawer:** Real-time quantity adjustments (`+` / `-`), subtotal and total calculations, item removal, and an animated free shipping threshold progress bar (`₹999`). - **Wishlist Toggle using ES6** `Set` **:** Manages product bookmarking with live badge counter updates. - **Quick View Modal & Toast Notifications:** Pop-up product details modal and non-intrusive alert toasts.

```
// Excerpt from EXP12/Casestudy/2/script.js
function addToCart(productId) {
  const item = products.find(p => p.id === productId);
  if (!item) return;

  const existing = cart.find(c => c.id === productId);
  if (existing) {
    existing.qty += 1;
  } else {
    cart.push({ ...item, qty: 1 });
  }

  updateCartUI();
  showToast(`Added "${item.name}" to cart! 🪔`);
  toggleCart(true);
}
```

Case Study 2 Screenshot



Screenshot 2026-10-26 232611.png

Result / Conclusion

Experiment 12 was successfully implemented and verified. The practical demonstrated the construction of reactive Single-Page Applications: managing dynamic DOM states with jQuery, synchronizing in-memory application models with `localStorage`, ingesting external REST APIs via asynchronous AJAX calls, and architecting advanced modular shopping cart systems.